

[声瀚嵌入式端 ECNR 算法库接口文档]

用户手册

文件编号:	文件版本: V1.0.3	发布/日期: 2021/10/23
拟制/日期: 2021/03/01	审核/日期: 2021/09/27	批准/日期: 2021/10/23
受控状态:	印章位置	密级: ☒ Low ☐ Middle ☐ Top
分发号:	发布对象: ☒ 已签署NDA客户 ☐ 其他: _____	
电子文件存放位置:		
地址: 上海市浦东新区松涛路647弄3号楼102室 电话: 021-55666043		



上海声瀚信息科技有限公司
未经许可不得扩散

变更纪录

版本	更新内容（章节号、概要）	更新时间	拟制人	审核人	批准人	批准日期
V1.0.0	初始基础版本	2021/03/01	张超鹏			
V1.0.1	增加通话模式	2021/05/16	周伟林			
V1.0.2	根据同步需求修改个别 API 接口	2021/09/27	周伟林	顾樑	蔡洪滨	2021/10/23
V1.0.3	修改接口加入本地在线授权功能	2022/07/19		陈学超		

非常感谢您采用我司的产品，如果您有什么疑问或需要请随时联系我们。

本手册可能包含技术上不准确的地方、或与软件例子代码不相符的地方、或印刷错误。我司将根据软件 SDK 的功能增强而更新本手册的内容，并将定期改进或更新本手册所描述的信息。更新的内容将在本手册的新版本中加入，恕不另行通知。

用户手册

1. 引言

1.1 编写的目的

本文是上海声瀚信息科技有限公司（以下简称“声瀚”）发布的基于嵌入式硬件平台的离线 ECNR 语音处理（包括回声消除、语音增强及降噪等）SDK 用户手册，用于指导用户基于本 SDK 的二次开发工作。

1.2 SDK 简介

声瀚语音处理 SDK 是上海声瀚信息科技有限公司基于语音处理方面的软件开发工具包，基于声瀚多年的语音研发成果。包括软件的平台支持库文件、相应的 C 语言接口描述头文件，适用于嵌入式平台的各种语音产品二次开发，提供的功能集包括语音增强（集成回声消除（AEC）、声学降噪（NR））等部分，用户基于本 SDK 可方便进行开发。

1.3 定义

语音处理：包括降噪、声学环境回声消除等音频信号处理

1.4 适用范围

目前本 SDK 适用于多种嵌入式处理器平台；

1.5 授权机制

本 SDK 的使用由上海声瀚信息科技有限公司授权，未经授权用户，请勿使用或传播。已授权用户使用时，请保持本 SDK 文件的完整性，禁止对 SDK 进行逆向工程及修改。

有关授权事宜，请联系上海声瀚信息科技有限公司程刚。

电话：

邮箱：chengg@uvoicetech.com.

2. SDK 版本更新

2.1 当前版本 V1.0.2

v1.0.0

2021/03/01

1. 初始版本
3. 审核确认 API 接口信息

v1.0.1

2021/05/16

1. 优化 API 接口，增加通话模式下的信号处理功能

v1.0.2

2021/09/27

1. 处理模式改成同步模式，修改了一个 API 接口

v1.0.3

2022/07/19

1. 修改调用接口，加入本地设备的在线授权功能

....

3. SDK 使用说明

3.1 SDK 简介

声瀚 SDK 用于对麦克风采集到的语音进行声学处理，支持多达 8 路麦克风信号及 2 路音频的回采信号，以实现语音的降噪、唤醒、回声消除等处理。

算法的包含文件：

uvoice_ecnr_api.h

uvoice_license.h

以及一个 SDK 动态（或静态）库文件。

3.2 算法 SDK 函数使用说明

1. 算法库句柄创建

```
void* UvEcnr_Create (UvEcnr_InputMode in_mod,
    UvEcnr_OutputMode out_mode, int sample_rate,void* param);
```

功能说明：

本函数主要用于算法库的初始化操作。

参数说明：

参数	说明	备注
in_mod	输入音频的排布格式，如： 1mic1ref(单麦克风单参考)、 1mic2ref(双麦克风单参考) ... 等	支持多种输入模式，但正式发布的算法库根据客户具体需求，一般会锁定到用户需要的输入模式。
out_mode	输出音频的模式，目前只支持两种模式： 1. uvoice_ecnr_mode_phone，通话模式，更注重输出音频的听感； 2. uvoice_ecnr_mode_asr，识别模式，更注重控制输出音频的失真状况，更有利于识别	正式发布的算法库根据用户的具体需求，一般会锁定到具体的输出模式。
Sample_rate	待处理音频的采样率	缺省为 16KHz 采样，一般会根据客户需求锁定特定采样率后发布
param	授权相关的结构体指针	参考头文件
Return 值	算法库控制句柄	不可修改

输入输出模式的定义如下：

```
typedef enum _UvEcnr_InputMode_{
    uvoice_ecnr_mode_1mic1ref = 0,
    uvoice_ecnr_mode_1mic2ref,
    uvoice_ecnr_mode_2mic1ref,
    uvoice_ecnr_mode_2mic2ref,
    uvoice_ecnr_mode_3mic1ref,
    uvoice_ecnr_mode_3mic2ref,
    uvoice_ecnr_mode_4mic1ref,
    uvoice_ecnr_mode_4mic2ref,
    uvoice_ecnr_mode_1mic0ref,
    uvoice_ecnr_mode_2mic0ref,
}UvEcnr_InputMode;
```

输出模式定义如下:

```
typedef enum _UvEcnr_OutputMode_{
    uvoice_ecnr_mode_phone = 0,
    uvoice_ecnr_mode_asr,
}UvEcnr_OutputMode;
```

授权相关的结构体定义如下:

```
typedef struct uv_activate_param
{
    char *license;
    char *license_path_file;
    char *uuid;
    int activate_type;
    uvoice_auth_callback_t auth_cb;
}uv_activate_param;
```

其中:

license 字符串为声瀚发布给用户的授权码, 原则上一台设备一条授权码;

license_path_file 为本地存放授权信息的路径或地址(如为文件, 需包括完整文件名);

uuid 指针存放用户设备的唯一 id, 必须保证唯一性, id 重复的多台设备将无法成功获得授权;

activate_type: 记录授权信息的类型, 0: 表示以文件方式记录, 1: flash, 2: efuse;

uvoice_auth_callback_t 函数指针类型定义如下(以实际发布的头文件定义为准);

auth_cb: 回调函数地址, 该回调函数用来向声瀚授权服务器发送本地信息并进行授权校验;
函数定义如下:

```
typedef const char* (*uvoice_auth_callback_t)(const char* https_post_body);
```

这个回调函数需要用户自行实现, 声瀚算法库授权模块会调用此函数, 该函数需要把输入参数发送给声瀚授权服务器, 并返回授权校验信息。

声瀚授权服务器地址为: https://srv01.51asr.com:8007/asrsn_active2

这个回调函数的行为应该和以下 curl 命令等效:

```
curl -k -s --data-binary $https_post_body https://srv01.51asr.com:8007/asrsn_active2
```

实现样例可参考 demo code.

2. 获取算法库授权状态

int UvEcnr_GetAuthError();

参数说明：无输入参数

返回参数：

0：授权成功

1：授权参数出错

4：设备已经激活过

5：不正确的授权码

6：授权码使用次数超限

7：授权码已被其他设备占用

其他返回值：其他错误

功能说明：

可以在 UvEcnr_Create 之后调用，检查授权状态

3. 获取算法库的版本号信息

const char* UvEcnr_GetVersion (void)

参数说明：无需输入参数

功能说明：

本函数用于校验算法库的版本号

4. 算法库的初始化

uvoice_ecnr_status UvEcnr_Init (void* handle)

功能说明：

参数说明：

参数	说明	备注
void *handle	算法库控制句柄	由 Create 函数创建
Return 值	初始化后算法库状态	

功能说明：

本函数用于初始化算法库

算法库状态包括：

typedef enum uvoice_ecnr_status{

UV_ECNR_GENER_ERROR = -1,

UV_ECNR_UNINIT_ERROR = -2,

UV_ECNR_NULLPTR_ERROR = -3,

UV_ECNR_PARAMETER_ERROR = -4,

UV_ECNR_VERIFY_ERROR = -5,

UV_ECNR_PARAMETER_WARNING = 1,

算法库出现运行错误

算法库未初始化

算法库控制句柄为 NULL

发现输入参数错误

发现数据校验错误

输入参数可能不规范

```
UV_ECNR_FILE_OPEN_FAILED = -6,          资源文件不能打开
UV_ECNR_OK = 0,                          算法库处于正常运行状态
}uvoice_ecnr_status;
```

5. 算法库状态重置

```
uvoice_ecnr_status UvEcnr_Reset(void* hdl);
```

功能说明：

参数说明：

参数	说明	备注
void *handle	算法库控制句柄	由 Create 函数创建
Return 值	算法库状态	

功能说明：

某些情况下，如音频发生了重大突变，可能会影响到算法收敛状况的，需要对算法库的状态进行重置。一般情况下，不需要执行该操作。

6. 算法库处理调用

```
uvoice_ecnr_status UvEcnr_Process(void* handle, uv_ecnr_audio_buf *audio,  
signed short** ppOut);
```

功能说明：

本函数用于向算法输入音频数据，获取处理后音频数据。

参数说明：

参数	说明	备注
void *handle	算法库控制句柄	
uv_ecnr_audio_buf* audio_buf	音频数据指针，多通道（包括麦克风音频和参考音频）分块排列。 struct audio_buf { short *audioin[8]; short *audioref[2]; };	缺省模式下，每个块均为采样频率 16K，采样精度 16bit，帧长 160 个采样点(10ms)，short 类型的数据
signed short** ppOut	返回存放输出音频的地址信息	缺省模式下为 16KHz 采样，160 个采样点（10ms），单通道，short 类型的数据
Return 值	算法库当前运行状态	无异常则继续运行

7. SDK 库资源释放

```
void UvEcnr_Destroy(void* hdl);
```


功能说明：

本函数用于释放语音处理对象。

参数说明：

参数	说明	备注
void *handle	算法库控制句柄	

3.3 示例代码：

请参考 demo.c